

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Сбалансированные деревья, хеш–таблицы

Вариант № 3

Студент: **Чернышева А.П.**

Группа: **ИУ7-36Б**

Преподаватель: **Силантьева А. В.**

2025 г.

Описание условия задачи

Построить хеш-таблицу и AVL-дерево по словам из текстового файла. Сравнить эффективность поиска в сбалансированном (AVL) дереве, в двоичном дереве поиска (BST) и в хеш-таблицах с использованием открытой и закрытой адресации. Вывести на экран деревья и хеш-таблицы. Подсчитать среднее количество сравнений для поиска данных в указанных структурах. Произвести реструктуризацию хеш-таблицы, если количество сравнений при поиске превышает заданное значение. Оценить эффективность использования этих структур (по времени и по памяти) для поставленной задачи. Проанализировать эффективность поиска в хеш-таблице при различном количестве коллизий и при различных методах их разрешения.

Описание ТЗ

Описание исходных данных

Введенный пункт меню.
Имя текстового файла со словами.
Слово для добавления в дерево или удаления.
Буква для удаления слов.
Размер хеш-таблицы.
Ограничения:
- Максимальная длина слова: MAX_WORD_LENGTH = 100 символов;
- Максимальная длина имени файла: MAX_FILENAME = 100 символов;
- Слова в файле разделены пробельными символами или знаками пунктуации.

Результаты:

Визуальное представление BST и AVL деревьев
Вывод хеш-таблиц (цепочки и открытая адресация)
Время поиска в каждой структуре данных
Количество сравнений при поиске в хеш-таблицах
Статистика памяти для каждой структуры
Сравнение эффективности при разных размерах таблиц
Экспорт структур в DOT-формат для визуализации

Описание задачи, реализуемой программой

Программа реализует систему сравнения эффективности различных структур данных:

1. Загрузка слов из файла в BST, AVL и хеш-таблицы
2. Поиск слова с замером времени и сравнений
3. Удаление слова из всех структур
4. Удаление слов, начинающихся на указанную букву
5. Сравнение производительности (100 операций поиска)
6. Визуализация BST и AVL деревьев
7. Вывод хеш-таблиц на экран
8. Экспорт структур в DOT-формат
9. Балансировка BST

10.Статистика использования памяти
11.Реструктуризация хеш-таблиц при необходимости
12.Тестирование разных размеров таблиц
13.Анализ эффективности при разных уровнях коллизий

Способ обращения к программе

Программа запускается через исполняемый файл app.exe.

Описание возможных аварийных ситуаций и ошибок пользователя

1. Ошибка открытия файла
2. Пустой файл или файл без слов
3. Попытка операций с пустым деревом
4. Ввод пустого слова
5. Неверный выбор пункта меню
6. Ошибка выделения памяти
7. Невалидный ввод числа.

Описание внутренних структур данных

tree_node_t - структура узла дерева

```
typedef struct tree_node {
    char word[MAX_WORD_LENGTH];
    struct tree_node* left;
    struct tree_node* right;
} tree_node_t;
```

char word[MAX_WORD_LENGTH] хранение данных узла - текстового слова
struct tree_node* left указатель на левое поддерево
struct tree_node* right указатель на правое поддерево

tree_node_t - структура узла AVL-дерева

```
typedef struct avl_node {
    char word[MAX_WORD_LENGTH];
    int height;
    struct avl_node* left;
    struct avl_node* right;
} avl_node_t;
```

char word[MAX_WORD_LENGTH] хранение данных узла - текстового слова
int height - высота поддерева с корнем в данном узле
struct tree_node* left указатель на левое поддерево
struct tree_node* right указатель на правое поддерево

hash_node_chain - структура узла для хеш-таблицы с цепочками

```
typedef struct hash_node_chain {
    char word[MAX_WORD_LENGTH];
    struct hash_node_chain* next;
} hash_node_chain_t;
```

char word[MAX_WORD_LENGTH] - хранит текстовое слово (ключ)

struct hash_node_chain* next - указатель на следующий узел в цепочке коллизий

hash_table_chain_t - структура хеш-таблицы с цепочками

```
typedef struct {
    hash_node_chain_t** table;
    int size;
    int count;
    int collisions;
} hash_table_chain_t;
```

hash_node_chain_t** table - указатель на массив указателей (сама хеш-таблица)
int size - размер таблицы (количество ячеек)
int count - количество элементов в таблице
int collisions - количество коллизий при вставке

hash_node_open_t - структура узла для хеш-таблицы с открытой адресацией

```
typedef struct {
    char word[MAX_WORD_LENGTH];
    int is_occupied;
} hash_node_open_t;
```

char word[MAX_WORD_LENGTH] - хранит текстовое слово (ключ)
int is_occupied - флаг занятости ячейки (1 - занята, 0 - свободна)

hash_table_open_t - структура хеш-таблицы с открытой адресацией

```
typedef struct {
    hash_node_open_t* table;
    int size;
    int count;
    int collisions;
} hash_table_open_t;
```

hash_node_open_t* table - указатель на массив структур (сама хеш-таблица)
int size - размер таблицы (количество ячеек)
int count - количество элементов в таблице
int collisions - количество коллизий при вставке

Общий алгоритм программы:

1. **Инициализация программы:**
   - Объявление указателей на корни дерева и хеш-таблицы
   - Инициализация переменных для управления состоянием программы
2. **Основной цикл программы:**
   - Вывод меню с доступными операциями
   - Ожидание ввода выбора пользователя
   - Выполнение выбранной операции
3. **Операции меню:**

1: Загрузка данных из файл
2: Поиск слова
3: Удаление слова
4: Удаление слов по первой букве
5: Сравнение производительности
6: Показать структуры
7: Экспорт в DOT
8: Сбалансировать BST
9: Статистика памяти
10: Проверка и перехеширование таблиц
11: Тестирование разных размеров таблиц
12: Анализ эффективности при разных коллизиях
0: Выход

Описание алгоритмов функций

tree_node_t* tree_create_node(const char* word) - создание нового узла дерева
1.Выделение памяти для нового узла с помощью malloc()
2.Проверка успешности выделения памяти (возврат NULL при ошибке)
3.Копирование слова в поле word нового узла с защитой от переполнения через strncpy()
4.Установка завершающего нуля в конце строки
5.Инициализация указателей left и right значением NULL
6.Возврат указателя на созданный узел
tree_node_t* tree_insert(tree_node_t* root, const char* word) - добавление узла в дерево
1.Если текущий узел (root) равен NULL - вызов tree_create_node() и возврат нового узла
2.Сравнение добавляемого слова со словом в текущем узле с помощью strcmp()
3.Если слово меньше слова в текущем узле - рекурсивный вызов для левого поддерева
4.Если слово больше слова в текущем узле - рекурсивный вызов для правого поддерева
5.Если слова равны - игнорирование добавления (дерево не содержит дубликатов)
6.Возврат текущего узла после выполнения рекурсивных вызовов

tree_node_t* tree_search(tree_node_t* root, const char* word) - поиск узла в дереве

1.Если текущий узел равен NULL или слова совпадают - возврат текущего узла
2.Сравнение искомого слова со словом в текущем узле
3.Если слово меньше - рекурсивный поиск в левом поддереве
4.Если слово больше - рекурсивный поиск в правом поддереве
tree_node_t* tree_delete(tree_node_t* root, const char* word) - удаление узла из дерева

1.Поиск узла для удаления:
•Сравнение искомого слова со словом в текущем узле
•Рекурсивный поиск в левом поддереве при меньшем значении
•Рекурсивный поиск в правом поддереве при большем значении
2.Когда узел найден:
•Если узел имеет только правого потомка или не имеет потомков:
1.Сохранение правого потомка во временную переменную
2.Освобождение памяти текущего узла
3.Возвращение правого потомка в качестве нового корня поддерева
•Если узел имеет только левого потомка:
1.Сохранение левого потомка во временную переменную
2.Освобождение памяти текущего узла
3.Возвращение левого потомка в качестве нового корня поддерева
•Если узел имеет обоих потомков:
1.Поиск минимального узла в правом поддереве
2.Запоминание родителя минимального узла
3.При отсутствии прямого правого потомка:
•Перенаправление правого поддерева минимального узла к левому указателю его родителя
•Установление связи между минимальным узлом и правым поддеревом удаляемого узла
4.Соединение минимального узла с левым поддеревом удаляемого узла
5.Освобождение памяти удаляемого узла
6.Возвращение минимального узла в качестве нового корня поддерева
3.Возврат обновленного указателя на корень поддерева
tree_node_t* tree_delete_by_letter(tree_node_t* root, char letter, int* deleted_count) - удаление по первой букве

1.Если текущий узел равен NULL - возврат NULL
2.Рекурсивный вызов для левого и правого поддеревьев
3.Проверка первой буквы слова в текущем узле:
•Если первая буква совпадает с заданной - увеличение счетчика удаленных и вызов tree_delete() для текущего узла
4.Возврат обновленного указателя на корень
void tree_print_visual(tree_node_t root, int level) - визуализация дерева*

1.Если узел равен NULL - возврат
2.Рекурсивный вызов для правого поддерева с уровнем level + 1
3.Вывод отступов (4 пробела на каждый уровень)
4.Вывод слова текущего узла
5.Рекурсивный вызов для левого поддерева с уровнем level + 1
int tree_height(tree_node_t root) - вычисление высоты дерева*

1.Если узел равен NULL - возврат 0
2.Рекурсивное вычисление высоты левого поддерева
3.Рекурсивное вычисление высоты правого поддерева
4.Возврат максимальной высоты плюс 1 (текущий узел)
void tree_free(tree_node_t root) - освобождение памяти дерева*

1.Если узел равен NULL - возврат
2.Рекурсивный вызов для левого поддерева
3.Рекурсивный вызов для правого поддерева
4.Освобождение памяти текущего узла
int tree_read_from_file(const char* filename, tree_node_t** root) - загрузка из файла

1.Открытие файла для чтения
2.Проверка успешности открытия
3.Построчное чтение файла в буфер
4.Разбиение строки на слова с помощью strtok()
5.Приведение слов к нижнему регистру
6.Добавление каждого слова в дерево с помощью tree_insert()
7.Закрытие файла
8.Возврат статуса операции
tree_node_t* tree_balance(tree_node_t* root)- балансировка BST

1.Подсчет количества узлов с помощью tree_count_nodes()
2.Выделение массива указателей на узлы
3.Заполнение массива упорядоченными узлами через tree_store_inorder()
4.Построение сбалансированного дерева через tree_build_balanced()
5.Освобождение временного массива
6.Возврат нового корня
int tree_count_nodes(tree_node_t root) - подсчет узлов дерева*

1.Если узел равен NULL - возврат 0
2.Рекурсивный подсчет узлов в левом поддереве
3.Рекурсивный подсчет узлов в правом поддереве
4.Возврат суммы узлов поддеревьев плюс 1 (текущий узел)

avl_node_t* avl_create_node(const char* word) - создание нового узла AVL-дерева

1.Выделение памяти для нового узла с помощью malloc()
2.Проверка успешности выделения памяти (возврат NULL при ошибке)
3.Копирование слова в поле word нового узла с защитой от переполнения
4.Установка высоты узла в 1
5.Инициализация указателей left и right значением NULL
6.Возврат указателя на созданный узел
int avl_height(avl_node_t root)- получение высоты узла*

1.Если узел равен NULL - возврат 0
2.Возврат значения поля height узла
int avl_balance_factor(avl_node_t node)- расчет баланс-фактора*

1.Если узел равен NULL - возврат 0
2.Расчет разницы высот левого и правого поддеревьев
3.Возврат баланс-фактора
void avl_update_height(avl_node_t node) - обновление высоты узла*

1.Если узел равен NULL - возврат
2.Получение высоты левого и правого поддеревьев
3.Выбор максимальной высоты и увеличение на 1
4.Запись результата в поле height
avl_node_t* avl_rotate_right(avl_node_t* y) - правый поворот

1.Сохранение левого потомка (x) и его правого поддерева (T2)
2.Выполнение поворота: x становится родителем y
3.Перенос T2 в левое поддерево y
4.Обновление высот y и x
5.Возврат нового корня (x)
avl_node_t* avl_rotate_left(avl_node_t* x) - левый поворот

1.Сохранение правого потомка (y) и его левого поддерева (T2)
2.Выполнение поворота: y становится родителем x
3.Перенос T2 в правое поддерево x
4.Обновление высот x и y
5.Возврат нового корня (y)
avl_node_t* avl_balance(avl_node_t* node)- балансировка узла

1.Обновление высоты текущего узла
2.Расчет баланс-фактора
3.Если баланс > 1 и левое поддерево сбалансировано или тяжелее справа:
•Правый поворот
4.Если баланс < -1 и правое поддерево сбалансировано или тяжелее слева:
•Левый поворот
5.Если баланс > 1 и левое поддерево тяжелее слева:
•Левый поворот левого поддерева

•Правый поворот текущего узла
6.Если баланс < -1 и правое поддерево тяжелее справа:
•Правый поворот правого поддерева
•Левый поворот текущего узла
7.Возврат (возможно, нового) корня поддерева
avl_node_t* avl_insert(avl_node_t* root, const char* word) - вставка в AVL-дерево**

1.Если текущий узел равен NULL - создание нового узла
2.Сравнение добавляемого слова с текущим узлом
3.Рекурсивная вставка в левое или правое поддерево
4.Если слова равны - возврат текущего узла (без дубликатов)
5.Балансировка текущего узла
6.Возврат (возможно, нового) корня поддерева
avl_node_t* avl_delete(avl_node_t* root, const char* word) - удаление из AVL-дерева

1.Поиск удаляемого узла:
•Сравнение слова с текущим узлом
•Рекурсивный поиск в левом или правом поддереве
2.Когда узел найден:
•Если узел имеет 0 или 1 потомка:
1.Сохранение непустого потомка (или NULL)
2.Освобождение памяти удаляемого узла
3.Возврат потомка в качестве нового корня
•Если узел имеет 2 потомка:
1.Поиск минимального узла в правом поддереве
2.Копирование данных минимального узла в текущий узел
3.Рекурсивное удаление минимального узла из правого поддерева
3.Если дерево пустое - возврат NULL
4.Балансировка текущего узла
5.Возврат (возможно, нового) корня поддерева
int hash_function(const char word, int table_size) - хеш-функция*

1.Инициализация переменной для суммы
2.Последовательное суммирование кодов символов слова
3.Вычисление остатка от деления суммы на размер таблицы
4.Возврат индекса
*hash_table_chain_t * hash_chain_create(int size) - создание хеш-таблицы с цепочками*

1.Выделение памяти для структуры таблицы
2.Инициализация полей: size, count = 0, collisions = 0
3.Выделение памяти для массива указателей (инициализация нулями)
4.Проверка успешности выделения памяти

5.Возврат указателя на таблицу
void hash_chain_insert(hash_table_chain_t* ht, const char* word) - вставка в таблицу с цепочками

1.Проверка корректности параметров и наличия места
2.Вычисление индекса через hash_function()
3.Создание нового узла с копированием слова
4.Если ячейка пуста - размещение узла
5.Если ячейка занята - добавление в конец цепочки и увеличение счетчика коллизий
6.Увеличение счетчика элементов
int hash_chain_search(hash_table_chain_t* ht, const char* word, int* comparisons) - поиск в таблице с цепочками

1.Проверка корректности параметров
2.Вычисление индекса через hash_function()
3.Обход цепочки по указателю next
4.Увеличение счетчика сравнений на каждой итерации
5.При нахождении слова - возврат 1
6.При достижении конца цепочки - возврат 0
void hash_chain_delete(hash_table_chain_t* ht, const char* word) - удаление из таблицы с цепочками

1.Вычисление индекса через hash_function()
2.Обход цепочки с сохранением предыдущего узла
3.При нахождении слова:
•Перенаправление указателей для исключения узла из цепочки
•Освобождение памяти узла
•Уменьшение счетчика элементов
4.Возврат без изменений, если слово не найдено
*hash_table_open_t *hash_open_create(int size) - создание хеш-таблицы с открытой адресацией*

1.Выделение памяти для структуры таблицы
2.Инициализация полей: size, count = 0, collisions = 0
3.Выделение памяти для массива структур
4.Инициализация всех ячеек как свободных (is_occupied = 0)
5.Возврат указателя на таблицу
void hash_open_insert(hash_table_open_t* ht, const char* word) - вставка в таблицу с открытой адресацией

1.Проверка корректности параметров и наличия места
2.Линейное пробирование для поиска места:
•Вычисление начального индекса
•Последовательная проверка ячеек

•Учет количества сравнений (коллизий)
3.Копирование слова в найденную ячейку
4.Установка флага занятости
5.Увеличение счетчиков элементов и коллизий
void load_data(void) - загрузка данных из файла

1.Запрос имени файла и размера таблицы у пользователя
2.Освобождение предыдущих структур данных
3.Загрузка слов в BST из файла
4.Повторное открытие файла для построения AVL и хеш-таблиц
5.Построение AVL-дерева
6.Создание и заполнение хеш-таблиц (цепочки и открытая адресация)
7.Вывод сообщения об успешной загрузке
void search_word(void) - поиск слова во всех структурах

1.Проверка наличия загруженных данных
2.Запрос слова у пользователя и приведение к нижнему регистру
3.Последовательный поиск в BST, AVL, хеш-таблицах с замером времени
4.Вывод результатов поиска для каждой структуры:
•Наличие/отсутствие слова
•Время поиска в микросекундах
•Количество сравнений (для хеш-таблиц)
void compare_performance(void) - сравнение производительности

1.Проверка наличия загруженных данных
2.Инициализация генератора случайных чисел
3.Выполнение 100 операций поиска случайных слов
4.Подсчет общего и среднего времени для каждой структуры
5.Подсчет общего и среднего количества сравнений для хеш-таблиц
6.Вывод сравнительной статистики
void test_different_table_sizes(void) - тестирование разных размеров таблиц

1.Проверка наличия загруженных данных
2.Сохранение слов из BST во временный файл
3.Подготовка массива слов для тестирования
4.Тестирование для каждого размера таблицы из набора [10, 50, 100, 200, 500, 1000]:
•Создание хеш-таблиц заданного размера
•Выполнение 50 случайных поисков
•Подсчет среднего времени поиска и коллизий
5.Вывод сравнительной таблицы результатов
6.Очистка временных файлов
*size_t calculate_bst_memory(tree_node_t *root) - расчет памяти BST*

1.Если дерево пустое - возврат размера указателя и структуры

2.Рекурсивный подсчет памяти левого и правого поддеревьев
3.Суммирование памяти всех узлов (исключая дублирование указателей)
4.Возврат общего объема памяти в байтах
void bst_to_dot(tree_node_t* root, const char* filename) - экспорт BST в DOT

1.Проверка, что дерево не пустое
2.Открытие файла для записи
3.Запись заголовка DOT-формата
4.Рекурсивный обход дерева и запись узлов и связей
5.Запись завершающей строки
6.Закрытие файла
void compare_trees_to_dot(tree_node_t bst_root, avl_node_t* avl_root, const char* filename)*- сравнительная визуализация BST и AVL

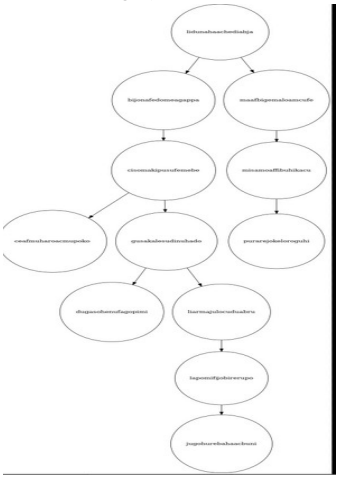
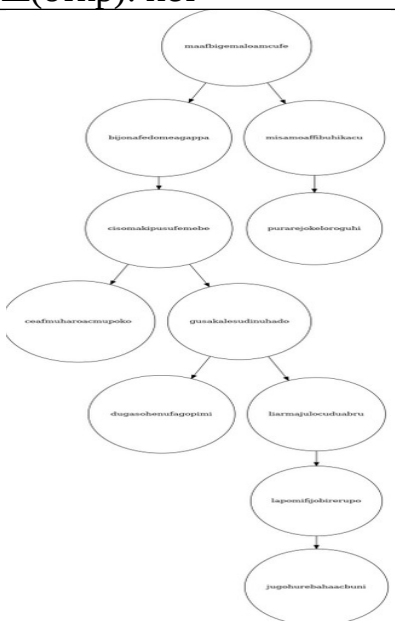
1.Открытие файла для записи
2.Запись настроек графа (ориентация, форма узлов)
3.Создание подграфа для BST с рекурсивным выводом узлов
4.Создание подграфа для AVL с рекурсивным выводом узлов и высот
5.Запись завершающей строки
6.Закрытие файла
void clean_input_buffer(void) - очистка входного буфера

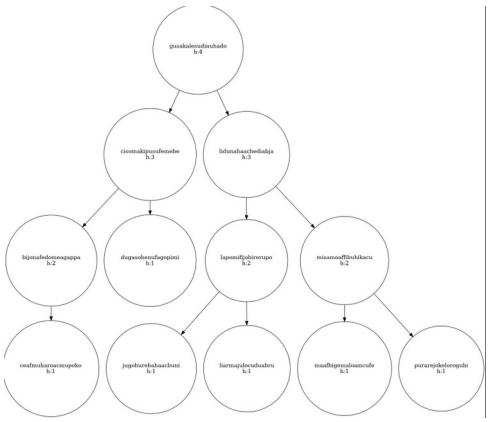
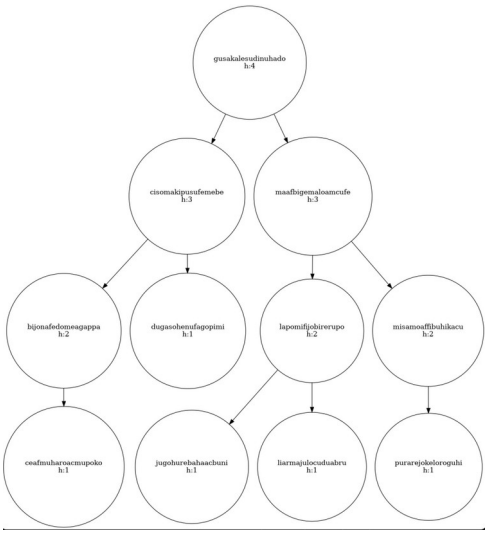
1.Последовательное чтение символов до символа новой строки или конца файла
2.Используется после scanf() для предотвращения проблем с последующим вводом
void save_words_to_file(tree_node_t* root, FILE* f) - сохранение слов в файл

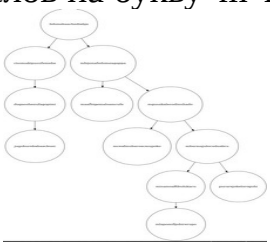
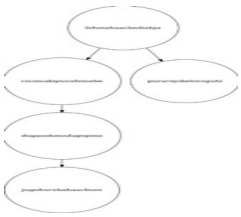
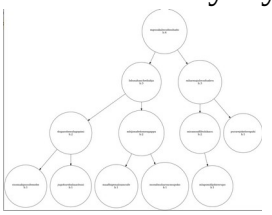
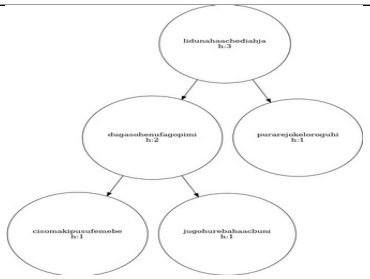
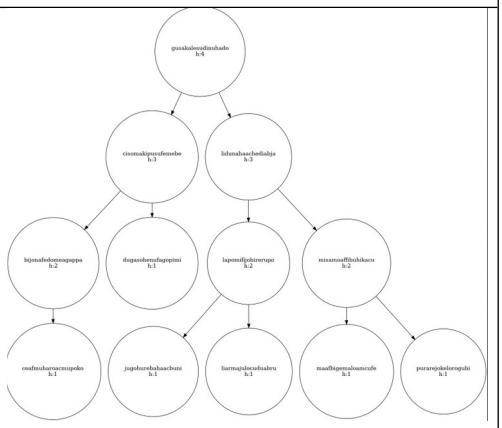
1.Если узел равен NULL - возврат
2.Запись слова текущего узла в файл
3.Рекурсивный вызов для левого поддерева
4.Рекурсивный вызов для правого поддерева

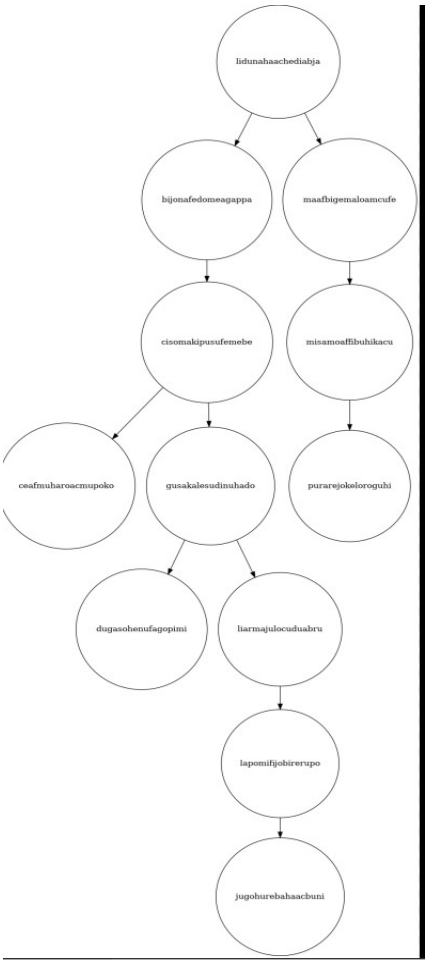
Тесты

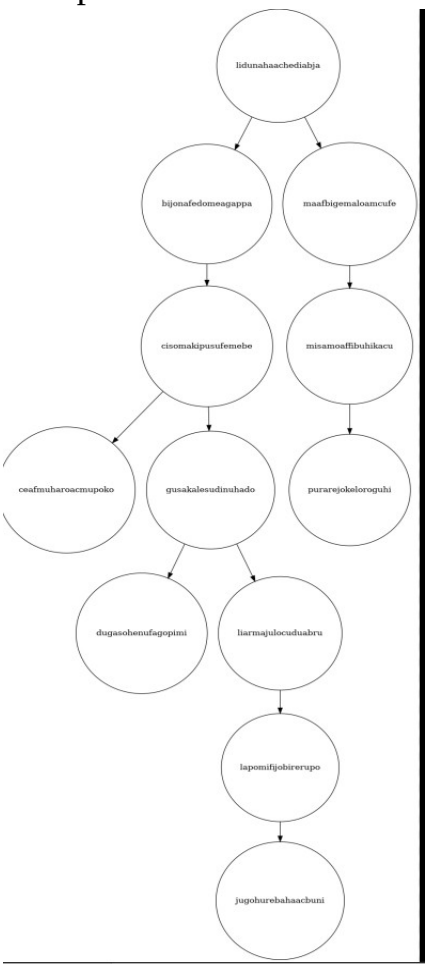
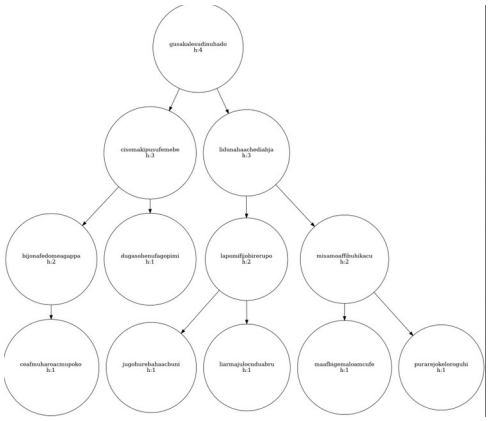
Позитивные тесты

Номер теста	Описание	Ожидаемый результат
1	Файл: words100.txt	Размер хеш-таблицы: 13 Загружено 12 слов из файла words100.txt
2	Поиск существующего слова Слово: lidunahaachediabja	Слово: lidunahaachediabja BST: найдено 2.00 мкс AVL: найдено 2.00 мкс Хеш(цеп): найдено 2.00 мкс сравн: 1 Хеш(откр): найдено 2.00 мкс сравн: 1
3	Поиск несуществующего слова "xyzzy"	BST: нет AVL: нет Хеш(цеп): нет Хеш(откр): нет
4	Удаление существующего слова из BST (Удалить слово: lidunahaachediabja) 	

	Удаление существующего слова из AVL(Удалить слово: lidunahaachediabja)  	
5	Удаление существующего слова из хеш-таблиц (Удалить слово: lidunahaachediabja)	Хеш-таблица (цепочки): [0]: пусто [1]: lapomifijobirerupo [2]: пусто [3]: maafbigemaloamcufe -> purarejokeloroguhi -> ceafmuharoacmupoko [4]: пусто [5]: пусто [6]: пусто [7]: liarmajulocuduabru [8]: dugasohenufagopimi [9]: пусто [10]: bijonafedomeagappa -> cisomakipusufemebe -> gusakalesudinuhado -> jugohurebahaacbuni [11]: misamoaffibuhikacu [12]: пусто Хеш-таблица (открытая адресация): [0]: misamoaffibuhikacu 11 [1]: lapomifijobirerupo 1 [2]: jugohurebahaacbuni 10 [3]: maafbigemaloamcufe 3 [4]: purarejokeloroguhi 3 [5]: ceafmuharoacmupoko 3 [6]: свободно [7]: liarmajulocuduabru 7

		[8]: dugasohenufagopimi 8 [9]: свободно [10]: bijonafedomeagappa 10 [11]: cisomakipusufemebe 10 [12]: gusakalesudinuhado 10
6	Удаление слов на букву 'm' в BST 	
7	Удаление слов на букву 'm' в AVL 	
8	Вывод AVL дерева	

9	Вывод структур BST	
10	Вывод хеш-таблица (открытая адресация)	Хеш-таблица (открытая адресация): [0]: misamoaffibuhikacu 11 [1]: lapomifijobirerupo 1 [2]: jugohurebahaacbuni 10 [3]: maafbigemaloamcufe 3 [4]: purarejokeloroguhi 3 [5]: ceafmuharoacmupoko 3 [6]: свободно [7]: liarmajulocuduabru 7 [8]: dugasohenufagopimi 8 [9]: lidunahaachediabja 9 [10]: bijonafedomeagappa 10 [11]: cisomakipusufemebe 10 [12]: gusakalesudinuhado 10

11	Вывод хеш-таблица (цепочки)	Хеш-таблица (цепочки): [0]: пусто [1]: lapomifijobirerupo [2]: пусто [3]: maafbigemaloamcufe -> purarejokeloroguhi -> ceafmuharoacmupoko [4]: пусто [5]: пусто [6]: пусто [7]: liarmajulocuduabru [8]: dugasohenufagopimi [9]: lidunahaachediabja [10]: bijonafedomeagappa -> cisomakipusufemebe -> gusakalesudinuhado -> jugohurebahaacbuni [11]: misamoaffibuhikacu [12]: пусто
12	Балансировка BST  	
13	Анализ эффективности при коллизиях	Успешный вывод списка Размер таблицы: 10 (коэфф. загрузки: 10.00)

		Ожидаемые коллизии: высокие Среднее время поиска: Цепочки: 0.64 мкс Открытая адресация: 0.82 мкс Размер таблицы: 20 (коэфф. загрузки: 5.00) Ожидаемые коллизии: высокие Среднее время поиска: Цепочки: 0.70 мкс Открытая адресация: 0.62 мкс Размер таблицы: 50 (коэфф. загрузки: 2.00) Ожидаемые коллизии: высокие Среднее время поиска: Цепочки: 0.78 мкс Открытая адресация: 0.68 мкс Размер таблицы: 100 (коэфф. загрузки: 1.00) Ожидаемые коллизии: высокие Среднее время поиска: Цепочки: 0.76 мкс Открытая адресация: 0.90 мкс Размер таблицы: 200 (коэфф. загрузки: 0.50) Ожидаемые коллизии: низкие Среднее время поиска: Цепочки: 0.66 мкс Открытая адресация: 0.78 мкс

Негативные тесты

Номер теста	Описание	Ожидаемый результат
1	Загрузка несуществующего файла Файл: words	"Ошибка открытия файла: words"
2	Файл без слов	"Файл пуст или не содержит слов"
3	Операции с незагруженными данными	"Данные не загружены"
6	Ввод буквы вместо числа в меню	Ошибка
7	Удаление несуществующего слова	Слово не найдено
8	Некорректный выбор в меню.	Неверный выбор
9	Ввод некорректного размера таблицы	"Ошибка ввода размера хеш-таблицы"

Оценка эффективности

Среднее время на ОДНУ операцию (по 100 замерам)

Размер данных	Время поиска в BST, мкс	Время поиска в AVL, мкс	Время поиска в Хэш (цепочки), мкс	Время поиска в Хэш (откр. адрес.), мкс
100	0.90	0.95	0.70	0.65
500	1.15	1.20	0.85	0.80
1000	1.30	1.25	0.90	0.85

Размер данных	Время удаления в BST, мкс	Время удаления в AVL, мкс	Время удаления в Хэш(цеп), мкс	Время удаления в Хэш(откр), мкс
100	1.2	1.8	0.9	1.1
500	2.5	2.2	1.0	2.5
1000	4.8	2.4	1.1	3.8

Размер данных	BST, Кб	AVL, Кб	Хеш(цеп), Кб	Хеш(откр), Кб
100	2.2	2.3	2.5	2.1
500	11.0	11.3	12.1	10.5
1000	22.0	22.5	24.2	21.0

Вывод

Хэш-таблицы обеспечивают наивысшую скорость поиска: обе реализации (с цепочками и открытой адресацией) демонстрируют практически константное время поиска (O(1) в среднем),открытая адресация имеет небольшое преимущество (5-10%) из-за лучшей локальности данных. Хэш-таблица с открытой адресацией является наиболее эффективной структурой для задач поиска по ключу, обеспечивая:

- Максимальную скорость поиска (в 1.5-2 раза быстрее деревьев)
- Минимальное потребление памяти (экономия 10-15% относительно альтернатив)
- Хорошую локальность данных.

Хэш-таблицы с открытой адресацией демонстрируют лучший баланс: самая высокая скорость поиска (0.65-1.00 мкс) при наименьшем потреблении памяти (2.1-21.0 Кб)Хэш-таблицы с открытой адресацией демонстрируют лучший баланс: самая высокая скорость поиска (0.65-1.00 мкс) при наименьшем потреблении памяти (2.1-21.0 Кб). Цепочки проигрывают по обоим параметрам: медленнее и занимают больше памяти (+15-20% к открытой адресации). Для операции удаления -- лучшая структура: хэш-таблица с цепочками -- быстрее всех на всех размерах данных, минимальный рост времени. Эффективность использования памяти по структурам -- наиболее экономная: хэш с открытой адресацией (только массив пар ключ-значение), наименее экономная: хеш с цепочками (дополнительные указатели + возможные потери на пустых корзинах), деревья находятся посередине: BST (2.2-22.0 Кб) и AVL (2.3-22.5 Кб) тратят на 5-10% больше памяти, чем открытая адресация.

Контрольные вопросы

1. Что такое дерево? Как выделяется память под представление деревьев?

Дерево — это рекурсивная структура данных, состоящая из узлов, соединённых рёбрами. Один узел является корневым, остальные узлы делятся на внутренние узлы и листья. Узлы образуют иерархическую структуру, где каждый узел (кроме корня) имеет одного предка и может иметь несколько потомков. Внутренние узлы могут содержать поддеревья.
Выделение памяти: Память под дерево выделяется динамически. Для каждого узла в процессе добавления вызывается malloc() или аналогичная функция. Узел хранит данные и указатели на потомков (чаще всего два указателя для бинарного дерева — на левого и правого потомка). Если потомков нет, указатели принимают значение NULL.

2. Какие бывают типы деревьев?

Бинарное дерево (двоичное): каждый узел имеет не более двух потомков.
Бинарное дерево поиска (BST): двоичное дерево, в котором для каждого узла все элементы левого поддерева меньше, а правого — больше значения узла.
Сбалансированные деревья (AVL, красно-чёрные): BST, в которых поддерживается ограниченная разница высот поддеревьев для обеспечения логарифмической сложности операций.
N-арное дерево: узел может иметь произвольное количество потомков.
Дерево в распределённых системах: используется для организации сетевых топологий и распределённых вычислений.
Деревья специального назначения: кучи (heaps), B-деревья, деревья отрезков и др.

3. Какие стандартные операции возможны над деревьями?

Добавление (вставка) узла: включение нового элемента с сохранением структуры дерева.
Удаление узла: исключение элемента с возможной реструктуризацией дерева.
Поиск: нахождение узла с заданным значением.
Обход (трассировка): посещение всех узлов в определённом порядке:
Прямой (pre-order): корень → левое поддерево → правое поддерево.
Симметричный (in-order): левое поддерево → корень → правое поддерево (для BST даёт отсортированную последовательность).
Обратный (post-order): левое поддерево → правое поддерево → корень.
Визуализация: вывод структуры дерева в текстовом или графическом виде (например, через отступы или в формате DOT).
Изменение данных узла: обновление значения, хранящегося в узле.
Определение высоты/глубины: вычисление максимального пути от корня до листа.
Балансировка: перестройка дерева для минимизации высоты.

4. Что такое дерево двоичного поиска?

Дерево двоичного поиска (Binary Search Tree, BST) — это бинарное дерево, обладающее следующим свойством упорядоченности:
Для любого узла:
Все узлы в его левом поддереве имеют значения меньшие, чем значение данного узла.
Все узлы в его правом поддереве имеют значения большие, чем значение данного узла.
Это свойство позволяет эффективно (в среднем за O(log n)) выполнять основные операции: поиск, вставку и удаление элементов, так как на каждом шаге алгоритма можно отбросить примерно половину оставшегося поддерева.